Danh sách đề tài môn Blockchain và ứng dụng

1. **Truy xuất nguồn gốc dược phẩm**
   - **Mục tiêu:** Hiểu cách Blockchain đảm bảo minh bạch, chống hàng giả trong chuỗi cung ứng dược phẩm.
   - **Nội dung thực hiện:**
     - Thiết kế struct LoThuoc gồm: id, tên thuốc, nhà sản xuất, khối lượng, và một danh sách các mốc hành trình (mỗi mốc gồm địa điểm, thời gian, ghi chú).
     - Viết hàm taoLoThuoc(tenThuoc, soLuong) để nhà sản xuất khởi tạo một lô thuốc mới.
     - Viết hàm ghiNhanMocHanhTrinh(loId, diaDiem, ghiChu) cho phép các bên trong chuỗi (nhà phân phối, nhà thuốc) bổ sung một mốc mới vào lịch sử lô thuốc — dữ liệu cũ không được sửa hay xoá.
     - Viết hàm xemLichSuLo(loId) trả về toàn bộ các mốc đã ghi nhận theo đúng thứ tự thời gian.
     - Viết script Python (web3.py): tạo 2 lô thuốc, ghi đầy đủ 4 mốc (sản xuất → vận chuyển → kiểm định → giao nhà thuốc) cho lô 1, chỉ 2 mốc đầu cho lô 2 (đang vận chuyển), sau đó truy vấn và in lịch sử của cả 2 lô.
     - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
   - **Kết quả đạt được:**
     - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan xác minh được).
     - File Notebook (.ipynb) chạy đầy đủ, in ra đúng lịch sử của lô thuốc 1 (4 mốc) và lô thuốc 2 (2 mốc, đang dở dang) — chứng minh dữ liệu cũ không bị mất hay sửa đổi.
     - Báo cáo ngắn (2–3 trang) phân tích lợi ích minh bạch/chống làm giả và hạn chế (chi phí gas, phụ thuộc dữ liệu nhập tay) khi áp dụng Blockchain cho ngành dược.

2. **Quản lý minh bạch hợp đồng thuê nhà**
   - **Mục tiêu:** Ứng dụng Blockchain giảm tranh chấp trong hợp đồng thuê nhà nhờ tính minh bạch, bất biến.
   - **Nội dung thực hiện:**
     - Thiết kế struct HopDong gồm: chủ nhà, người thuê, tiền thuê/tháng, tiền cọc, ngày bắt đầu, ngày kết thúc, trạng thái (DangHieuLuc/DaGiaHan/DaChamDut).
     - Viết hàm kyHopDong(nguoiThue, tienThue, tienCoc, thoiHan) để chủ nhà khởi tạo hợp đồng.

- Viết hàm xacNhanNguoiThue(hopDongId) — chỉ đúng địa chỉ người thuê được ghi trong hợp đồng mới gọi được, dùng để xác nhận đồng ý điều khoản.
- Viết hàm giaHanHopDong(hopDongId, soThangThem) và hàm chamDutHopDong(hopDongId, soTienHoanCoc) — cho phép chấm dứt hợp đồng và ghi rõ số tiền cọc được hoàn lại kèm lý do.
- Viết script mô phỏng đủ 3 tình huống: (1) ký và xác nhận hợp đồng, (2) gia hạn thêm 6 tháng, (3) chấm dứt hợp đồng và hoàn một phần tiền cọc — in trạng thái hợp đồng trước/sau mỗi bước.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

- **Kết quả đạt được:**
  - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
  - Notebook chạy đủ 3 tình huống (ký, gia hạn, chấm dứt) với bằng chứng trạng thái hợp đồng thay đổi đúng sau mỗi bước.
  - Báo cáo phân tích: những điều khoản nào nên đưa lên Blockchain, điều khoản nào (ví dụ tình trạng thực tế căn nhà) vẫn cần xác minh ngoài đời.

3. **Minh bạch hồ sơ bồi thường bảo hiểm y tế**
   - **Mục tiêu:** Hiểu cách Blockchain hạn chế gian lận trong xét duyệt bồi thường bảo hiểm.
   - **Nội dung thực hiện:**
     - Thiết kế struct HoSo gồm: mã hồ sơ, người yêu cầu, số tiền yêu cầu, loại dịch vụ (mô tả chung, không ghi thông tin bệnh án cụ thể), trạng thái (ChoDuyet/DaDuyet/TuChoi), lý do (nếu từ chối).
     - Viết hàm nopHoSo(soTienYeuCau, loaiDichVu) để người yêu cầu nộp hồ sơ mới.
     - Viết hàm duyetHoSo(hoSoId) và tuChoiHoSo(hoSoId, lyDo) — chỉ địa chỉ ví của đơn vị bảo hiểm (onlyInsurer) mới được gọi 2 hàm này.
     - Viết hàm xemTrangThai(hoSoId) trả về đầy đủ thông tin và trạng thái hiện tại của 1 hồ sơ.
     - Viết script mô phỏng: nộp 4 hồ sơ với số tiền khác nhau, duyệt 3 hồ sơ, từ chối 1 hồ sơ kèm lý do cụ thể, sau đó in bảng trạng thái toàn bộ 4 hồ sơ.
     - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
   - **Kết quả đạt được:**
     - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
     - Notebook chạy đủ 4 hồ sơ với cả 2 nhánh duyệt/từ chối, có bảng tổng hợp trạng thái rõ ràng.

- Báo cáo phân tích ưu điểm (không thể sửa quyết định sau khi đã duyệt/từ chối) và hạn chế (dữ liệu y tế nhạy cảm không nên lưu trực tiếp on-chain) của giải pháp.

4. **Hóa đơn điện tử chống gian lận thuế**
   - **Mục tiêu:** Ứng dụng Blockchain tăng minh bạch, chống làm giả hóa đơn điện tử.
   - **Nội dung thực hiện:**
     - Thiết kế struct HoaDon gồm: mã hóa đơn (duy nhất), người bán, người mua, giá trị, thời gian phát hành.
     - Viết hàm phatHanhHoaDon(maHoaDon, nguoiMua, giaTri) — bắt buộc require mã hóa đơn CHƯA từng tồn tại, nếu trùng phải revert.
     - Viết hàm traCuuHoaDon(maHoaDon) trả về đầy đủ thông tin hóa đơn nếu tồn tại.
     - Viết hàm kiemTraTrungLap(maHoaDon) trả về true/false.
     - Viết script mô phỏng: phát hành 5 hóa đơn hợp lệ với mã khác nhau, sau đó thử phát hành lại đúng 1 mã đã tồn tại — in ra bằng chứng giao dịch bị revert (không tạo được hóa đơn giả/trùng).
     - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
   - **Kết quả đạt được:**
     - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
     - Notebook chứng minh rõ ràng: 5 hóa đơn hợp lệ được tạo, giao dịch tạo hóa đơn trùng mã bị hệ thống từ chối (có log lỗi/revert reason).
     - Báo cáo phân tích khả năng chống làm giả hóa đơn so với hệ thống hóa đơn điện tử tập trung hiện nay.

5. **Chống gian lận đơn hàng trong thương mại điện tử**
   - **Mục tiêu:** Hiểu cách Blockchain ghi nhận minh bạch, không thể sửa đổi vòng đời một đơn hàng online.
   - **Nội dung thực hiện:**
     - Thiết kế enum TrangThai gồm 4 giá trị theo đúng thứ tự: DaDatHang, DaThanhToan, DaGiao, DaNhan; struct DonHang gồm: người mua, người bán, mô tả sản phẩm, trạng thái hiện tại.
     - Viết hàm taoDonHang(nguoiBan, moTaSanPham) để khởi tạo đơn ở trạng thái DaDatHang.
     - Viết hàm capNhatTrangThai(donHangId, trangThaiMoi) — bắt buộc kiểm tra trạng thái mới phải là bước KẾ TIẾP hợp lệ (không cho nhảy cóc, không cho lùi lại), nếu sai phải revert.
     - Viết hàm xemLichSuTrangThai(donHangId) trả về trạng thái hiện tại và toàn bộ mốc thời gian đã chuyển.

- Viết script mô phỏng: đơn hàng 1 chuyển tuần tự đúng cả 4 bước; đơn hàng 2 thử nhảy thẳng từ DaDatHang sang DaNhan (phải bị revert) — in kết quả cả 2 trường hợp.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook chứng minh đơn hàng 1 hoàn tất đúng trình tự, đơn hàng 2 bị chặn đúng khi cố nhảy bước.
- Báo cáo phân tích lợi ích khi khách hàng và người bán cùng nhìn thấy một lịch sử đơn hàng không thể chỉnh sửa một phía.

6. **Truy xuất nguồn gốc cà phê xuất khẩu**

• **Mục tiêu:** Minh bạch hóa hành trình nông sản từ nông trại đến cảng xuất khẩu.

• **Nội dung thực hiện:**
- Thiết kế struct LoCaPhe gồm: id, tên nông trại, khối lượng, và danh sách các mốc hành trình (mỗi mốc gồm tên công đoạn, địa điểm, thời gian).
- Viết hàm taoLoHang(nongTrai, khoiLuong) khởi tạo lô cà phê.
- Viết hàm ghiNhanMocHanhTrinh(loId, tenCongDoan, diaDiem) để bổ sung từng mốc (thu hoạch → sơ chế → đóng gói → xuất khẩu) khi lô hàng di chuyển qua từng công đoạn.
- Viết hàm xemHanhTrinh(loId) trả về toàn bộ các mốc theo đúng thứ tự đã ghi nhận.
- Viết script mô phỏng: tạo 1 lô cà phê, ghi nhận đủ 4 mốc hành trình, sau đó truy vấn và in lại toàn bộ hành trình theo thứ tự thời gian.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook in đúng và đầy đủ 4 mốc hành trình của lô cà phê theo thứ tự thời gian ghi nhận.
- Báo cáo phân tích giá trị của truy xuất nguồn gốc đối với nông sản xuất khẩu (uy tín thương hiệu, đáp ứng yêu cầu thị trường nhập khẩu).

7. **Vé sự kiện chống làm giả bằng Blockchain**

• **Mục tiêu:** Ngăn vé giả, vé bán trùng bằng cách phát hành và xác thực vé trên Blockchain.

• **Nội dung thực hiện:**

- Thiết kế struct Ve gồm: id, tên sự kiện, địa chỉ chủ vé, trạng thái đã sử dụng (bool).
- Viết hàm phatHanhVe(suKien, nguoiMua) để tạo vé mới, mặc định đã sử dụng = false.
- Viết hàm checkIn(veId) — bắt buộc require vé CHƯA được sử dụng, sau khi gọi thì đánh dấu đã sử dụng = true; nếu gọi lại lần 2 phải revert với thông báo rõ ("Ve da duoc su dung").
- Viết hàm xemTrangThaiVe(veId) trả về thông tin và trạng thái hiện tại.
- Viết script mô phỏng: phát hành 6 vé, check-in 4 vé, thử check-in lại 1 trong 4 vé đã dùng (phải revert), thử check-in 1 mã vé không tồn tại (phải revert) — in kết quả từng trường hợp.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook chứng minh chặn đúng cả 2 tình huống lỗi (vé dùng lại, vé không tồn tại) kèm log lỗi cụ thể.
- Báo cáo phân tích so sánh với hệ thống vé giấy/mã QR tập trung hiện nay.

8. **Quản lý minh bạch hoạt động hiến máu**

• **Mục tiêu:** Minh bạch hóa việc ghi nhận và sử dụng túi máu hiến tặng.

• **Nội dung thực hiện:**
- Thiết kế enum TrangThaiMau gồm: DaHien, DaNhapKho, DaSuDung; struct DonViMau gồm: id, nhóm máu, trạng thái, nơi sử dụng (nếu có).
- Viết hàm ghiNhanHienMau(nhomMau) tạo 1 đơn vị máu mới ở trạng thái DaHien.
- Viết hàm nhapKho(donViId) chuyển trạng thái sang DaNhapKho.
- Viết hàm ghiNhanSuDung(donViId, noiSuDung) chuyển trạng thái sang DaSuDung — bắt buộc phải qua đúng 2 bước trước đó (require trạng thái hiện tại phải là DaNhapKho).
- Viết script mô phỏng: ghi nhận 4 đơn vị máu, đưa 3 đơn vị đi qua đủ cả 3 bước, 1 đơn vị dừng lại ở bước nhập kho — in vòng đời từng đơn vị máu.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).

- Notebook in đúng vòng đời của 4 đơn vị máu, phân biệt rõ đơn vị đã sử dụng và đơn vị còn tồn kho.
- Báo cáo phân tích lợi ích minh bạch hóa đối với hệ thống ngân hàng máu.

9. **Quản lý bản quyền và phân phối doanh thu nhạc số**
   - **Mục tiêu:** Minh bạch hóa bản quyền và tự động phân chia doanh thu theo lượt phát nhạc số.
   - **Nội dung thực hiện:**
     - Thiết kế struct BaiHat gồm: id, tên bài hát, danh sách đồng tác giả kèm tỉ lệ chia doanh thu (%), số dư đã tích lũy của từng tác giả.
     - Viết hàm dangKyBaiHat(tenBaiHat, danhSachTacGia[], danhSachTiLe[]) — bắt buộc require tổng tỉ lệ của tất cả tác giả phải đúng bằng 100%.
     - Viết hàm ghiNhanLuotPhat(baiHatId) payable — nhận một khoản doanh thu (ETH) mô phỏng cho 1 lượt phát, tự động cộng vào số dư từng tác giả đúng theo tỉ lệ đã đăng ký.
     - Viết hàm rutDoanhThu(baiHatId) để 1 tác giả rút phần doanh thu đang có trong số dư của mình.
     - Viết script mô phỏng: đăng ký 1 bài hát với 2 đồng tác giả (tỉ lệ 60/40), gửi 3 khoản doanh thu mô phỏng với giá trị khác nhau, mỗi tác giả rút doanh thu và kiểm tra số tiền nhận được có đúng tỉ lệ đã đăng ký hay không.
     - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
   - **Kết quả đạt được:**
     - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
     - Notebook chứng minh bằng số liệu cụ thể: tổng doanh thu 3 lượt phát được chia đúng 60/40 giữa 2 tác giả.
     - Báo cáo phân tích ứng dụng cho ngành công nghiệp âm nhạc số so với cách phân phối doanh thu qua bên trung gian hiện nay.

10. **NFT xác thực quyền sở hữu tác phẩm nghệ thuật số**
    - **Mục tiêu:** Ứng dụng NFT (ERC-721) để chứng minh quyền sở hữu tác phẩm số gốc.
    - **Nội dung thực hiện:**
      - Chuẩn bị metadata JSON (tên tác phẩm, tác giả, mô tả, link ảnh) và upload lên IPFS qua NFT.storage/Pinata để lấy link dạng ipfs://<cid>.
      - Thiết kế smart contract NFT (kế thừa chuẩn ERC-721) với hàm mintTacPham(chuSoHuu, tokenURI).
      - Viết hàm chuyenNhuong(nguoiNhan, tokenId) để chuyển quyền sở hữu NFT sang ví khác.

- Viết hàm xemChuSoHuuHienTai(tokenId) trả về địa chỉ ví đang sở hữu và tokenURI gắn với NFT.
- Viết script mô phỏng: mint 2 NFT cho 2 tác phẩm khác nhau (mỗi NFT trỏ tới 1 metadata JSON thật đã upload), chuyển nhượng 1 NFT sang ví thứ hai, sau đó tra cứu lại chủ sở hữu và mở link IPFS để xác nhận nội dung JSON đọc được.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng NFT triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook chứng minh mint thành công 2 NFT, chuyển nhượng đúng chủ sở hữu, và link IPFS mở được, hiển thị đúng nội dung metadata đã upload.
- Báo cáo mô tả ứng dụng NFT trong việc xác thực quyền sở hữu tác phẩm số, phân biệt với việc chỉ lưu file trên máy chủ thông thường.

11. **DApp bỏ phiếu bầu cử nội bộ minh bạch**

• **Mục tiêu:** Ứng dụng Blockchain đảm bảo bầu cử nội bộ minh bạch, không thể sửa kết quả.

• **Nội dung thực hiện:**
- Thiết kế struct UngVien gồm: tên, số phiếu hiện tại; dùng mapping địa chỉ ví → đã bầu hay chưa.
- Viết hàm themUngVien(ten) — chỉ ban tổ chức (onlyOwner) được thêm ứng viên, phải thêm trước khi mở bầu cử.
- Viết hàm boPhieu(ungVienId) — bắt buộc require cử tri (msg.sender) CHƯA bầu lần nào, sau khi bầu đánh dấu đã bầu = true và cộng phiếu cho ứng viên tương ứng.
- Viết hàm congBoKetQua() trả về danh sách toàn bộ ứng viên kèm số phiếu hiện tại, sắp theo số phiếu giảm dần.
- Viết script mô phỏng: thêm 3 ứng viên, mô phỏng 8 cử tri (8 ví khác nhau) bỏ phiếu, thử cho 1 cử tri bỏ phiếu lần thứ hai (phải bị revert), công bố và in kết quả cuối cùng.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook chứng minh 8 phiếu bầu hợp lệ được ghi nhận đúng, 1 lượt bầu trùng bị chặn (có log lỗi).
- Báo cáo phân tích tính minh bạch, khả năng kiểm chứng độc lập kết quả so với hình thức bầu cử thủ công/giấy.

12. **Minh bạch hóa quỹ hoạt động tổ chức phi lợi nhuận**
    - **Mục tiêu:** Công khai minh bạch nguồn thu và khoản chi của một tổ chức phi lợi nhuận.
    - **Nội dung thực hiện:**
        - Thiết kế struct GiaoDich gồm: loại (Thu/Chi), số tiền, mục đích/người đóng góp, thời gian ghi nhận.
        - Viết hàm ghiNhanKhoanThu(nguoiDongGop) payable để nhận đóng góp và cộng vào số dư quỹ.
        - Viết hàm ghiNhanKhoanChi(soTien, mucDich) — bắt buộc require số dư quỹ hiện tại đủ để chi, nếu không đủ phải revert.
        - Viết hàm xemSoDuQuy() và xemLichSuGiaoDich() trả về số dư hiện tại và toàn bộ lịch sử giao dịch theo thứ tự thời gian.
        - Viết script mô phỏng: 5 khoản thu từ 5 người đóng góp khác nhau, 3 khoản chi có mục đích rõ ràng, kiểm tra số dư cuối cùng có khớp đúng tổng thu trừ tổng chi; thử 1 khoản chi vượt số dư còn lại (phải revert).
        - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
    - **Kết quả đạt được:**
        - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
        - Notebook chứng minh số dư quỹ luôn khớp với lịch sử giao dịch, và khoản chi vượt quỹ bị chặn đúng.
        - Báo cáo phân tích lợi ích minh bạch tài chính đối với các bên đóng góp/tài trợ.

13. **Xác thực nguồn gốc và chống đạo văn tài liệu học thuật**
    - **Mục tiêu:** Dùng Blockchain lưu dấu thời gian và mã băm (hash) tài liệu để chứng minh nguồn gốc.
    - **Nội dung thực hiện:**
        - Thiết kế struct TaiLieu gồm: hash tài liệu (bytes32), tiêu đề, tác giả, thời gian nộp.
        - Viết hàm nopTaiLieu(hashTaiLieu, tieuDe) — bắt buộc require hash CHƯA từng tồn tại trong hệ thống (đây chính là cơ chế phát hiện trùng lặp).
        - Viết hàm kiemTraTrung(hashTaiLieu) trả về true/false, và nếu trùng thì trả kèm thông tin tài liệu gốc (tác giả, thời gian nộp đầu tiên).
        - Viết hàm tính hash SHA-256 của một file văn bản trong Python (dùng thư viện hashlib) trước khi gửi lên hợp đồng.
        - Viết script mô phỏng: nộp 5 tài liệu với nội dung/hash khác nhau, sau đó thử nộp một tài liệu có nội dung giống hệt tài liệu đã nộp trước nhưng khai tên tác giả khác (hash trùng → phải bị revert, hiển thị rõ tác giả và thời gian của tài liệu gốc).

- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

- **Kết quả đạt được:**
  - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
  - Notebook chứng minh phát hiện đúng tài liệu trùng hash và trả về đúng thông tin tác giả gốc.
  - Báo cáo phân tích giới hạn của phương pháp (chỉ phát hiện được khi nội dung giống hệt, không phát hiện được đạo văn diễn giải lại).

14. **Gây quỹ cộng đồng minh bạch (crowdfunding có hoàn tiền)**
    - **Mục tiêu:** Ứng dụng Blockchain đảm bảo minh bạch gọi vốn và tự động hoàn tiền nếu không đạt mục tiêu.
    - **Nội dung thực hiện:**
      - Thiết kế struct ChienDich gồm: mục tiêu số tiền, tổng số tiền đã góp, hạn chót (timestamp), đã rút tiền hay chưa; dùng mapping địa chỉ người góp → số tiền đã góp cho từng chiến dịch.
      - Viết hàm taoChienDich(mucTieu, hanChot) để khởi tạo chiến dịch mới.
      - Viết hàm gopVon(chienDichId) payable — cộng dồn số tiền góp của người gửi.
      - Viết hàm rutTien(chienDichId) — chỉ chủ chiến dịch được gọi, bắt buộc require đã đạt mục tiêu VÀ đã qua hạn chót.
      - Viết hàm hoanTien(chienDichId) — cho phép từng người góp tự rút lại đúng số tiền mình đã góp, chỉ khi đã qua hạn chót MÀ chưa đạt mục tiêu.
      - Viết script mô phỏng 2 kịch bản riêng biệt: (1) chiến dịch đạt mục tiêu → chủ chiến dịch rút tiền thành công; (2) chiến dịch không đạt mục tiêu → từng người góp tự hoàn tiền, kiểm tra số dư ví trước/sau mỗi kịch bản.
      - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
    - **Kết quả đạt được:**
      - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
      - Notebook chạy đúng cả 2 kịch bản (đạt và không đạt mục tiêu), có bằng chứng số dư ví thay đổi chính xác ở mỗi kịch bản.
      - Báo cáo phân tích so với các nền tảng gọi vốn cộng đồng tập trung hiện nay (Kickstarter...).

15. **Quản lý mượn/trả sách điện tử minh bạch**
    - **Mục tiêu:** Quản lý minh bạch quyền mượn/trả tài liệu số trong thư viện.
    - **Nội dung thực hiện:**

- Thiết kế struct Sach gồm: tiêu đề, tổng số bản copy, số bản đang được mượn; dùng mapping (sachId, người mượn) → hạn trả (timestamp).
- Viết hàm themSach(tieuDe, soLuong) để thủ thư thêm đầu sách mới.
- Viết hàm muonSach(sachId) — bắt buộc require còn ít nhất 1 bản copy trống, sau đó gán hạn trả = thời điểm hiện tại + 7 ngày.
- Viết hàm traSach(sachId) — so sánh thời điểm trả thực tế với hạn trả đã lưu để xác định có trả trễ hay không, và giải phóng 1 bản copy.
- Viết hàm xemTrangThaiMuon(sachId, nguoiMuon) trả về hạn trả và tình trạng (đang mượn/đã trả/trả trễ).
- Viết script mô phỏng: mượn 4 lượt cho 3 đầu sách khác nhau — 2 lượt trả đúng hạn, 1 lượt trả trễ (giả lập bằng cách chỉnh thời gian test tăng thêm 10 ngày), 1 lượt vẫn đang mượn; in bảng trạng thái toàn bộ 4 lượt.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook thể hiện rõ cả 3 trạng thái mượn sách (đúng hạn/trễ hạn/đang mượn) với dữ liệu cụ thể.
- Báo cáo phân tích ứng dụng cho hệ thống thư viện số của trường.

16. **Chia sẻ chi phí nhóm du lịch minh bạch**

• **Mục tiêu:** Minh bạch hóa việc chia sẻ chi phí sinh hoạt chung giữa các thành viên trong nhóm.

• **Nội dung thực hiện:**
- Thiết kế struct KhoanChi gồm: người ứng tiền, số tiền, mô tả, danh sách địa chỉ các thành viên cùng chia khoản chi này.
- Viết hàm themKhoanChi(soTien, moTa, danhSachThanhVien[]) — tự động chia đều số tiền cho danh sách thành viên tham gia khoản chi đó, cộng dồn vào công nợ giữa từng cặp thành viên.
- Viết hàm xemSoDuMoiNguoi(diaChiThanhVien) trả về số tiền người đó đang nợ (âm) hoặc đang được nhận lại (dương), tổng hợp từ toàn bộ các khoản chi.
- Viết hàm thanhToanBu(nguoiNhan) payable để 1 thành viên thanh toán bù trừ công nợ cho thành viên khác.
- Viết script mô phỏng: 5 khoản chi khác nhau do 3 thành viên khác nhau ứng trước cho các nhóm nhỏ khác nhau trong chuyến đi, tính và in số dư (nợ/được nhận) của từng thành viên, sau đó thực hiện 1 giao dịch thanh toán bù và kiểm tra lại số dư.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**

- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook in đúng số dư của từng thành viên trước và sau khi thanh toán bù, khớp với lịch sử 5 khoản chi.
- Báo cáo phân tích so sánh với việc tự ghi chép/nhắc nợ thủ công giữa bạn bè.

17. **Điểm danh hội nghị/sự kiện chống gian lận**

- **Mục tiêu:** Ứng dụng Blockchain xác thực điểm danh, chống điểm danh hộ.
- **Nội dung thực hiện:**
  - Thiết kế struct SuKien gồm: tên sự kiện; dùng mapping (suKienId, địa chỉ người tham dự) → đã điểm danh hay chưa.
  - Viết hàm taoSuKien(ten) để ban tổ chức tạo sự kiện mới.
  - Viết hàm diemDanh(suKienId) — bắt buộc require msg.sender CHƯA điểm danh cho đúng sự kiện này, sau đó đánh dấu đã điểm danh.
  - Viết hàm demSoNguoiThamDu(suKienId) trả về tổng số người đã điểm danh.
  - Viết script mô phỏng: 10 người (10 ví khác nhau) điểm danh cho 1 sự kiện, thử cho 1 người điểm danh lần thứ hai (phải bị revert), đếm và in tổng số người tham dự hợp lệ.
  - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
- **Kết quả đạt được:**
  - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
  - Notebook chứng minh 10 lượt điểm danh hợp lệ được ghi nhận và 1 lượt điểm danh trùng bị chặn.
  - Báo cáo phân tích áp dụng cho điểm danh hội nghị/hội thảo quy mô lớn.

18. **Quản lý hợp đồng thuê xe tự lái minh bạch**

- **Mục tiêu:** Minh bạch hóa điều khoản, thời gian thuê, và xử lý phạt vi phạm hợp đồng thuê xe.
- **Nội dung thực hiện:**
  - Thiết kế struct HopDongThueXe gồm: người thuê, mã xe, ngày bắt đầu, ngày trả dự kiến, tiền cọc, đã trả xe hay chưa; định nghĩa mức phạt cố định cho mỗi ngày trả trễ.
  - Viết hàm thueXe(xeId, soNgayThue) payable — tính và ghi nhận tiền cọc theo số ngày thuê đăng ký.
  - Viết hàm traXe(hopDongId) — so sánh thời điểm trả xe thực tế với ngày trả dự kiến; nếu trả trễ thì tính tiền phạt = số ngày trễ × mức phạt/ngày, trừ vào tiền cọc trước khi hoàn lại phần còn dư cho người thuê.

- Viết hàm xemChiTietHopDong(hopDongId) trả về đầy đủ thông tin và số tiền đã hoàn lại (nếu đã trả xe).
- Viết script mô phỏng 2 kịch bản: (1) trả xe đúng hạn → hoàn đủ 100% tiền cọc; (2) trả xe trễ 2 ngày → bị trừ đúng số tiền phạt tương ứng trước khi hoàn cọc — in số tiền hoàn lại của cả 2 kịch bản.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook chứng minh số tiền hoàn cọc được tính đúng cho cả 2 kịch bản (đúng hạn/trễ hạn).
- Báo cáo phân tích ứng dụng cho các nền tảng cho thuê xe tự lái ngang hàng (peer-to-peer).

19. **Truy xuất nguồn gốc thực phẩm hữu cơ**

• **Mục tiêu:** Minh bạch hóa chuỗi cung ứng thực phẩm hữu cơ từ trang trại đến siêu thị.

• **Nội dung thực hiện:**
- Thiết kế struct LoNongSan gồm: id, loại sản phẩm, tên nông trại, có chứng nhận hữu cơ hay không (bool), và danh sách các mốc hành trình (công đoạn, địa điểm, thời gian).
- Viết hàm taoLoSanPham(loaiSanPham, nongTrai, coChungNhan) khởi tạo lô hàng mới.
- Viết hàm ghiNhanMoc(loId, congDoan, diaDiem) để bổ sung mốc hành trình khi lô hàng đi qua từng công đoạn (thu hoạch → kiểm định → vận chuyển → phân phối tại siêu thị).
- Viết hàm xemHanhTrinh(loId) trả về toàn bộ hành trình cùng thông tin chứng nhận hữu cơ.
- Viết script mô phỏng: tạo 2 lô hàng (1 lô có chứng nhận hữu cơ, 1 lô không), ghi nhận đủ 4 mốc hành trình cho mỗi lô, in ra so sánh hành trình và trạng thái chứng nhận của 2 lô.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook in đúng và đầy đủ hành trình của cả 2 lô hàng, phân biệt rõ lô có/không có chứng nhận hữu cơ.
- Báo cáo phân tích lợi ích đối với người tiêu dùng khi có thể tự tra cứu nguồn gốc thực phẩm.

20. **Đăng ký và xác thực quyền sở hữu trí tuệ mini (sáng chế/nhãn hiệu)**

- **Mục tiêu:** Ứng dụng Blockchain lưu dấu thời gian đăng ký ý tưởng/sáng chế để làm bằng chứng ưu tiên.
- **Nội dung thực hiện:**
  - Thiết kế struct YTuong gồm: hash (bytes32) của mô tả ý tưởng, mô tả ngắn công khai, người đăng ký, thời gian đăng ký.
  - Viết hàm dangKyYTuong(hash, moTaNgan) — bắt buộc require hash CHƯA từng tồn tại trong hệ thống.
  - Viết hàm traCuuUuTien(hash) trả về người đăng ký và thời gian đăng ký ĐẦU TIÊN của hash đó, nếu tồn tại.
  - Viết hàm Python tính hash SHA-256 từ nội dung mô tả chi tiết (giữ bí mật, không đưa lên chuỗi) trước khi gửi hash lên hợp đồng.
  - Viết script mô phỏng: 3 nhóm khác nhau đăng ký 3 ý tưởng khác nhau, sau đó mô phỏng 1 nhóm cố đăng ký lại đúng hash của một ý tưởng đã có trước đó (phải bị revert, đồng thời tra cứu ra được ai đã đăng ký trước và vào lúc nào).
  - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
- **Kết quả đạt được:**
  - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
  - Notebook chứng minh đúng thứ tự ưu tiên đăng ký theo thời gian và chặn được đăng ký trùng hash.
  - Báo cáo phân tích giới hạn (chỉ chứng minh THỜI ĐIỂM biết đến ý tưởng, không thay thế thủ tục đăng ký sở hữu trí tuệ chính thức).

21. **Quản lý chuỗi cung ứng linh kiện điện tử**

- **Mục tiêu:** Minh bạch hóa nguồn gốc linh kiện trong chuỗi cung ứng điện tử.
- **Nội dung thực hiện:**
  - Thiết kế struct LoLinhKien gồm: id, loại linh kiện, nhà cung cấp, số lượng, điểm đến lắp ráp, trạng thái (DangVanChuyen/DaLapRap).
  - Viết hàm taoLoLinhKien(loaiLinhKien, soLuong, nhaCungCap) khởi tạo lô hàng.
  - Viết hàm capNhatDiemDen(loId, diaDiem) ghi nhận lô hàng đã đến một điểm trung chuyển.
  - Viết hàm xacNhanLapRap(loId) đánh dấu lô linh kiện đã được lắp vào sản phẩm hoàn chỉnh.
  - Viết hàm xemNguonGoc(loId) trả về toàn bộ thông tin nguồn gốc và hành trình của lô hàng.
  - Viết script mô phỏng: tạo 3 lô linh kiện từ 3 nhà cung cấp khác nhau, mỗi lô đi qua 2 điểm trung chuyển trước khi được xác nhận lắp ráp, sau đó truy vấn nguồn gốc của cả 3 lô cùng lắp vào 1 sản phẩm.

- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

- **Kết quả đạt được:**
  - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
  - Notebook truy xuất đúng nguồn gốc và hành trình của cả 3 lô linh kiện.
  - Báo cáo phân tích ứng dụng kiểm soát chất lượng và trách nhiệm khi sản phẩm gặp lỗi.

22. **Xác thực xuất xứ hàng hiệu chống hàng giả**

- **Mục tiêu:** Ứng dụng Blockchain xác thực hàng chính hãng, chống hàng giả.
- **Nội dung thực hiện:**
  - Thiết kế struct SanPham gồm: mã định danh duy nhất (gắn với mã QR/NFC vật lý trên sản phẩm), nhà sản xuất, chủ sở hữu hiện tại.
  - Viết hàm dangKySanPham(maDinhDanh) — chỉ nhà sản xuất được gọi, bắt buộc require mã CHƯA từng tồn tại trong hệ thống.
  - Viết hàm chuyenNhuong(maDinhDanh, nguoiNhanMoi) — chỉ đúng chủ sở hữu hiện tại mới được gọi để chuyển quyền sở hữu (ví dụ khi bán lại hàng cũ).
  - Viết hàm xacThucSanPham(maDinhDanh) trả về sản phẩm có tồn tại trong hệ thống hay không, và nếu có thì trả về chủ sở hữu hiện tại.
  - Viết script mô phỏng: đăng ký 4 sản phẩm chính hãng, chuyển nhượng 1 sản phẩm qua 2 đời chủ, sau đó xác thực 1 mã sản phẩm không có trong hệ thống (kết luận là hàng giả) và 1 mã sản phẩm hợp lệ (trả về đúng chủ sở hữu hiện tại).
  - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
- **Kết quả đạt được:**
  - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
  - Notebook chứng minh phân biệt đúng sản phẩm chính hãng và sản phẩm không tồn tại (nghi hàng giả).
  - Báo cáo phân tích ứng dụng cho ngành hàng hiệu/hàng xa xỉ.

23. **Quản lý hồ sơ bảo trì thiết bị công nghiệp minh bạch**

- **Mục tiêu:** Minh bạch hóa lịch sử bảo trì, sửa chữa thiết bị công nghiệp.
- **Nội dung thực hiện:**
  - Thiết kế struct ThietBi gồm: tên thiết bị, danh sách các lần bảo trì (mỗi lần gồm ngày, nội dung công việc, tên kỹ thuật viên).
  - Viết hàm themThietBi(ten) để đăng ký 1 thiết bị mới vào hệ thống.

- Viết hàm ghiNhanBaoTri(thietBiId, noiDung, kyThuatVien) để bổ sung 1 lần bảo trì mới vào lịch sử — dữ liệu cũ không thể sửa/xoá.
- Viết hàm xemLichSuBaoTri(thietBiId) trả về toàn bộ các lần bảo trì theo đúng thứ tự thời gian.
- Viết script mô phỏng: đăng ký 2 thiết bị, ghi nhận 5 lần bảo trì cho thiết bị 1 (bởi 2 kỹ thuật viên khác nhau) và 2 lần cho thiết bị 2, in toàn bộ lịch sử bảo trì của cả 2 thiết bị.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook in đúng và đầy đủ lịch sử bảo trì của cả 2 thiết bị theo thứ tự thời gian.
- Báo cáo phân tích lợi ích đối với việc lập kế hoạch bảo trì định kỳ và xác định trách nhiệm khi thiết bị hỏng.

24. **Minh bạch hóa quy trình đấu thầu mua sắm công**

• **Mục tiêu:** Ứng dụng Blockchain tăng minh bạch trong đấu thầu mua sắm công.

• **Nội dung thực hiện:**
- Thiết kế struct GoiThau gồm: tên gói thầu, hạn nộp hồ sơ (timestamp), danh sách hồ sơ dự thầu (mỗi hồ sơ gồm nhà thầu, hash tài liệu, thời gian nộp), nhà thầu trúng thầu.
- Viết hàm taoGoiThau(ten, hanNop) để ban tổ chức tạo gói thầu mới.
- Viết hàm nopHoSo(goiThauId, hash) — bắt buộc require thời điểm nộp phải TRƯỚC hạn nộp, nếu nộp sau hạn phải revert.
- Viết hàm congBoKetQua(goiThauId, nhaThauTrung) — chỉ ban tổ chức được gọi, bắt buộc require đã QUA hạn nộp mới được công bố.
- Viết hàm xemDanhSachHoSo(goiThauId) trả về toàn bộ hồ sơ đã nộp kèm thời gian, phục vụ đối chiếu công khai.
- Viết script mô phỏng: 4 nhà thầu nộp hồ sơ đúng hạn, 1 nhà thầu thử nộp sau hạn (phải bị revert), sau khi qua hạn thì công bố kết quả chọn thầu và in danh sách toàn bộ hồ sơ hợp lệ.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook chứng minh hồ sơ nộp trễ bị từ chối, hồ sơ nộp đúng hạn không thể bị chỉnh sửa sau khi đã nộp.

- Báo cáo phân tích khả năng giảm tiêu cực trong đấu thầu công nhờ tính minh bạch, bất biến của dữ liệu.

25. **Minh bạch hóa quỹ học bổng sinh viên**
    - **Mục tiêu:** Công khai minh bạch quá trình xét duyệt và cấp phát học bổng.
    - **Nội dung thực hiện:**
        - Thiết kế struct DotCap gồm: tên đợt cấp, tổng quỹ, danh sách sinh viên đã được cấp (mỗi mục gồm sinh viên, số tiền).
        - Viết hàm taoDotCap(tenDot, tongQuy) payable để khởi tạo 1 đợt cấp học bổng với tổng ngân sách xác định.
        - Viết hàm capHocBong(dotId, sinhVien, soTien) — bắt buộc require tổng số tiền đã cấp cộng thêm khoản mới KHÔNG được vượt quá tổng quỹ, nếu vượt phải revert.
        - Viết hàm xemSoDuConLai(dotId) trả về số tiền quỹ còn lại chưa cấp.
        - Viết script mô phỏng: tạo 1 đợt cấp với tổng quỹ xác định, cấp học bổng cho 6 sinh viên với số tiền khác nhau, thử cấp cho sinh viên thứ 7 với số tiền vượt quá quỹ còn lại (phải revert), in số dư quỹ sau mỗi lần cấp.
        - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
    - **Kết quả đạt được:**
        - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
        - Notebook chứng minh quỹ không bao giờ bị cấp vượt mức, có bảng danh sách 6 sinh viên được cấp và số tiền tương ứng.
        - Báo cáo phân tích lợi ích minh bạch đối với đơn vị tài trợ học bổng.

26. **Quản lý minh bạch điểm rèn luyện và hoạt động ngoại khóa sinh viên**
    - **Mục tiêu:** Ứng dụng Blockchain ghi nhận minh bạch, không thể sửa điểm rèn luyện/hoạt động ngoại khóa.
    - **Nội dung thực hiện:**
        - Thiết kế struct HoatDong gồm: tên hoạt động, số điểm cộng tương ứng; dùng mapping (hoatDongId, sinh viên) → đã được ghi nhận tham gia hay chưa.
        - Viết hàm taoHoatDong(ten, diemCong) để đơn vị tổ chức tạo hoạt động mới.
        - Viết hàm ghiNhanThamGia(hoatDongId, sinhVien) — bắt buộc require sinh viên đó CHƯA được ghi nhận cho đúng hoạt động này (chống cộng điểm trùng).
        - Viết hàm xemTongDiem(sinhVien) trả về tổng điểm rèn luyện tích lũy từ tất cả hoạt động đã tham gia.
        - Viết script mô phỏng: tạo 5 hoạt động với mức điểm khác nhau, ghi nhận 4 sinh viên tham gia các tổ hợp hoạt động khác nhau, thử ghi

nhận trùng 1 sinh viên cho cùng 1 hoạt động (phải revert), in tổng điểm rèn luyện của cả 4 sinh viên.
    - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
- **Kết quả đạt được:**
    - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
    - Notebook in đúng tổng điểm rèn luyện của từng sinh viên, truy vết được từng hoạt động đã cộng điểm.
    - Báo cáo phân tích ứng dụng thay thế việc tổng hợp điểm rèn luyện thủ công hiện nay.

27. **Giao dịch tín chỉ năng lượng tái tạo mini**
- **Mục tiêu:** Mô phỏng thị trường giao dịch tín chỉ năng lượng tái tạo minh bạch trên Blockchain.
- **Nội dung thực hiện:**
    - Thiết kế mapping địa chỉ ví → số dư tín chỉ năng lượng tái tạo hiện có.
    - Viết hàm phatHanhTinChi(donViSanXuat, soLuong) — chỉ cơ quan cấp phép (onlyIssuer) được gọi, dùng để cấp tín chỉ mới cho 1 đơn vị sản xuất năng lượng sạch.
    - Viết hàm chuyenNhuongTinChi(nguoiNhan, soLuong) — bắt buộc require số dư người gửi đủ để chuyển, trừ/cộng số dư tương ứng ở cả 2 ví.
    - Viết hàm xemSoDuTinChi(vi) trả về số dư tín chỉ hiện tại của 1 địa chỉ ví.
    - Viết script mô phỏng: phát hành tín chỉ cho 2 đơn vị sản xuất năng lượng, thực hiện 4 giao dịch chuyển nhượng qua lại giữa 3 ví khác nhau (bao gồm cả bên mua tín chỉ để bù trừ phát thải), in số dư từng ví sau mỗi giao dịch.
    - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
- **Kết quả đạt được:**
    - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
    - Notebook chứng minh số dư tín chỉ mỗi ví luôn chính xác sau từng giao dịch chuyển nhượng.
    - Báo cáo phân tích ứng dụng của Blockchain trong thị trường tín chỉ carbon/năng lượng tái tạo thực tế.

28. **Quản lý minh bạch lịch sử sở hữu và bảo dưỡng xe cũ**
- **Mục tiêu:** Minh bạch hóa lịch sử sở hữu và bảo dưỡng để hỗ trợ mua bán xe cũ tin cậy.
- **Nội dung thực hiện:**

- Thiết kế struct Xe gồm: biển số, chủ sở hữu hiện tại, danh sách lịch sử sang tên (mỗi mục gồm chủ cũ, chủ mới, thời gian), danh sách lịch sử bảo dưỡng (mỗi mục gồm nội dung, thời gian).
- Viết hàm dangKyXe(bienSo, chuDauTien) để đăng ký 1 xe mới vào hệ thống với chủ sở hữu đầu tiên.
- Viết hàm sangTen(bienSo, chuMoi) — chỉ đúng chủ sở hữu hiện tại được gọi, ghi thêm 1 mục vào lịch sử sang tên.
- Viết hàm ghiNhanBaoDuong(bienSo, noiDung) để bổ sung 1 lần bảo dưỡng mới vào lịch sử.
- Viết hàm xemLichSuXe(bienSo) trả về đầy đủ cả lịch sử sở hữu và lịch sử bảo dưỡng theo đúng thứ tự thời gian.
- Viết script mô phỏng: đăng ký 1 xe, sang tên qua 3 đời chủ, xen kẽ ghi nhận 5 lần bảo dưỡng trong suốt quá trình, in toàn bộ lịch sử xe theo đúng trình tự thời gian thực tế đã xảy ra.
- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

• **Kết quả đạt được:**
- Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
- Notebook in đúng và đầy đủ lịch sử sở hữu (3 đời chủ) và lịch sử bảo dưỡng (5 lần) theo đúng thứ tự thời gian.
- Báo cáo phân tích lợi ích đối với người mua xe cũ khi có thể tự kiểm chứng lịch sử xe.

29. **Đăng ký và xác thực quyền sử dụng đất mini**

• **Mục tiêu:** Mô phỏng ứng dụng Blockchain trong đăng ký, xác thực quyền sử dụng đất, giảm tranh chấp.

• **Nội dung thực hiện:**
- Thiết kế struct ThuaDat gồm: mã số thửa đất, diện tích, chủ sở hữu hiện tại, danh sách lịch sử chuyển nhượng (mỗi mục gồm chủ cũ, chủ mới, thời gian).
- Viết hàm dangKyThuaDat(maSo, dienTich, chuDauTien) để đăng ký 1 thửa đất mới.
- Viết hàm chuyenNhuong(maSo, chuMoi) — bắt buộc require chỉ đúng chủ sở hữu hiện tại mới được gọi, nếu người gọi không phải chủ sở hữu phải revert.
- Viết hàm xemChuSoHuu(maSo) và xemLichSuChuyenNhuong(maSo) trả về chủ sở hữu hiện tại và toàn bộ lịch sử chuyển nhượng.
- Viết script mô phỏng: đăng ký 2 thửa đất, chuyển nhượng thửa đất 1 qua 2 lần hợp lệ, thử cho một địa chỉ KHÔNG phải chủ sở hữu cố tình chuyển nhượng thửa đất 2 (phải bị revert) — in kết quả cả 2 trường hợp.

- Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.

- **Kết quả đạt được:**
  - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
  - Notebook chứng minh chuyển nhượng hợp lệ thành công và chuyển nhượng trái phép bị chặn đúng.
  - Báo cáo phân tích khả năng ứng dụng cho hệ thống đăng ký đất đai, đồng thời nêu rõ giới hạn (không thay thế được cơ quan quản lý nhà nước).

30. **Truy xuất nguồn gốc hải sản đánh bắt**

- **Mục tiêu:** Minh bạch hóa nguồn gốc hải sản từ khai thác đến bàn ăn, hỗ trợ đánh bắt bền vững.
- **Nội dung thực hiện:**
  - Thiết kế struct LoHaiSan gồm: id, tên tàu khai thác, vùng biển, thời gian đánh bắt, danh sách các điểm phân phối đã đi qua (mỗi điểm gồm địa điểm, thời gian).
  - Viết hàm taoLoHaiSan(tenTau, vungBien) khởi tạo lô hải sản mới ngay khi tàu cập bến.
  - Viết hàm ghiNhanPhanPhoi(loId, diaDiem) để bổ sung 1 điểm phân phối mới khi lô hàng di chuyển (chợ đầu mối → nhà phân phối → điểm bán lẻ).
  - Viết hàm xemHanhTrinh(loId) trả về đầy đủ vùng biển đánh bắt và toàn bộ các điểm phân phối theo thứ tự thời gian.
  - Viết script mô phỏng: tạo 2 lô hải sản từ 2 tàu khác nhau (2 vùng biển khác nhau), mỗi lô đi qua 3 điểm phân phối, truy vấn và in hành trình đầy đủ của cả 2 lô.
  - Triển khai hợp đồng lên Sepolia testnet, ghi lại địa chỉ hợp đồng và link Etherscan.
- **Kết quả đạt được:**
  - Hợp đồng triển khai thành công trên Sepolia (có địa chỉ và link Etherscan).
  - Notebook in đúng và đầy đủ hành trình của cả 2 lô hải sản.
  - Báo cáo phân tích vai trò của giải pháp trong việc hỗ trợ xác minh đánh bắt hợp pháp, bền vững.